

Class exercise

Introduction

Why Bash?

Bash is a shell scripting language, its purpose is to operate the operating system: run programs, make the workflow with pipes, manage files, control processes.

In bioinformatics most of the softwares and tools are written in Perl, Python and R. **But for all of them, Bash is what gets you to the place where that analysis runs**, so like I mentioned in class "Bash is the Orchestra director". In this course you will learn how to be the orchestra director, not just the musicians, once you start to get comfortable with this, the sky is the limit.

In real high-throughput bioinformatics, the "opera theater" place is almost always a Linux-based computing cluster, accessed remotely, with no graphical interface at all.

This is a learning curve, so be patient and kind with yourself :white_heart::

This document took some examples from my former PI David Ray's from Texas Tech University class material, that he elaborated with his student Francisco Castellanos.

Assignment Goals:

1. Practice by your own, how to access to a terminal, and how to start doing basic tasks to start organizing your projects.
2. Practice the best practices for coding documentation.
3. Create your first script that will be scalable and reproducible to create all your directories and files for the Genomics and Transcriptomics Project.
4. Practice committing and pushing documents to Github
5. Explore and learn how to use Markdown.

1. Set up the environment:

Lets use your computer terminal!

Unless you have a linux operating system, our default terminal doesn't automatically speak bash. Even though a lot of commands overlap, they're not identical, and small differences can cause a script that works for you to fail for someone else, and remember we need our work be:

- Reproducible
- Scalable
- Shareable

So, let's make a few adjustments first.



IMPORTANT: These are temporary adjustments, in a real life situation you usually will have access to a Linux-based server or cluster.

Mac users:

Since macOS Catalina (2019), the default shell is zsh, not bash. Zsh and bash are like cousins, software engineers love to complicate our lives :woozy_face:.

But we can do a small adjustment to our terminal (no risks as this is going to be temporal).

Steps

1. Open the Terminal (Spotlight → search "Terminal").
2. Type `bash` and press Enter.
3. Type `echo $0` and press Enter, it will print the name of the language that you are currently using.

⚠ This only lasts for that terminal window/tab, when you close it, and open a new one, you'll be back in zsh by default. Just type bash again each session.

Windows users:

With windows is a liitle bit more complicated but not imposible. Windows doesn't ship with any Unix-style shell at all. So we need extra steps in comparisson with mac users:

Steps

We need to install a bash-only tool named Git Bash.

1. Go to [Git Bash website](#) and download Git for Windows.
2. Run the installer, accept the defaults (this also installs Git itself, which you'll want later for GitHub Desktop).
3. Once installed, open Git Bash from the Start menu.
4. You now have a real bash shell, every command from class will work here.

2. Create your first working directory

Where am I?

Looking at directories from within a terminal can seem confusing at first.

Our first interactions with a computer are 99% of the times through a graphical user interfaces (GUIs) like on Windows computers and Macs.

In this course however, most of the work will be done in a terminal, and might be a little abstract to understand at first. However, directories are exactly the same type of folders that you can see if you use any graphical file browser. Like when you click on "Downloads" or "Music" directories in your laptop.

Let's start by understanding where the work for this session will be taking place at. For this we need the command `pwd` (short for **p**rint **w**orking **d**irectory). This tells you the directory where you are currently working, this command is going to give you **the complete path!**

then we will use `ls` (short for **l**ist) to see what is inside that directory:

[!NOTE] Answer the following:  **Exercise 1:** Open your terminal and type the following commands.

In the coding box below, type what your terminal prompted:

#Command 1:

`pwd`

#Output:

`/Users/nchrste`

#-----

#Command 2:

`ls`

#Output:

`ba`

`Desktop Downloads Movies Pictures Documents Library Music Public`

#-----

#Command 3:

`ls -l`

#Output

`drwx-----+ 12 nchrste ACCOUNTS\Domain Users 384 Sep 2 15:27`

`Desktop`

`drwx-----+ 15 nchrste ACCOUNTS\Domain Users 480 Sep 9 17:32`

`Documents`

`drwx-----+ 301 nchrste ACCOUNTS\Domain Users 9632 Sep 9 18:01`

`Downloads`

`>drwx-----@ 85 nchrste ACCOUNTS\Domain Users 2720 Jun 25 15:10`

`Library`

`drwx----- 4 nchrste ACCOUNTS\Domain Users 128 Dec 23 2025`

`Movies`

`drwx-----+ 4 nchrste ACCOUNTS\Domain Users 128 Dec 23 2025`

`Music`

`drwx-----+ 4 nchrste ACCOUNTS\Domain Users 128 Dec 22 2025`

`Pictures`

`drwxr-xr-x+ 4 nchrste ACCOUNTS\Domain Users 128 Dec 22 2025`

`Public`

You have now learned the `pwd` and `ls` bash functions, you used `ls` followed by a flag `-l` which is called a flag or argument. These are prefixed with a hyphen (either `-` or `--`) and modify the preceding function, extending its functionality. Whereas a simple `ls` command lists the files within a directory, `-l` will display full details on every file contained in a directory.



You will eventually memorize these arguments. Certainly, not all of them. If you wanted to learn what other arguments are available for a function, you can simply ask for help using the `--help`

flag by typing: `ls --help`

Moving around

Now we want to go to our  Documents, for this we use the `cd` command (short for **c**hange **d**irectory). This is where you need to start tracking the Path and follow our best practices tips:

```
cd Documents/  
  
ls -l
```

Here you are going to see all the files that you have inside Documents.

? Did you notice that I did not write the entire path? (e.g. `/Users/dmoreno17/Documents/`) but just `Documents/`

Writing out the entire path every single time, all the way from the root, would be exhausting. This is where **relative paths** come in.

A relative path describes where something is starting from where you already are, instead of starting from the root.

Going back to David Rays' analogy we saw in class: if someone ask you where is restroom at Lo Shiavo, you would not begin your directions from the Milky Way. You would just say "the room in front of the elevators". That shortcut is a relative path.

The difference is small but important. An absolute path always starts with `/` and works from anywhere, while a relative path is written from your current location and only makes sense from there: `Documents/`

Organizing your files

Now that you understand paths, let's talk about something that usually even experienced bioinformaticians neglect doing: **keeping your files organized**.

When you run real analyses you will quickly pile up dozens of files, including raw datasets, scripts, intermediate files, final tables, and figures. If they all live together in one messy folder, sooner or later you will overwrite something important or lose track of what came from where.

The solution is simple, **maintain a consistent project structure**. Every project you build in this course, and every project you will later run should follow a similar skeleton the one we saw in class (Week 3):

For the rest of the semester, create a directory in your local computer named `btec_640`:

The basic directories that you should have in your local computer `btec_640` directory are:

-  class_exercises
-  genomics_project
-  transcriptomics_projects



Exercise 2:

For now let's create our `btec_640` directory inside `Documents/`, when you feel comfortable enough to move it around, you can put it wherever makes sense for you.

To create our own directory we will use `mkdir` (short for **make directory**). Just like with `ls`, we will use a **flag**, the `-p` flag lets you create a whole nested path in one go and quietly does nothing if the folder is already there.

To create a directory inside `Documents/`:

```
pwd #This is just sanity check to se that we are already inside Documents
mkdir -p btec_640/class_excercises
```

[!NOTE] Answer the following:

1. Move inside `btec_640` and list all the files and directories that are inside that directory.

Write down the files or directories listed:

Type your answer here:

```
class_excercises
genomics_project
transcriptomics_projects
```

2. You should be inside `btec_640` (not sure? type `pwd`).

Go back to `Documents/` using `cd ../`. This command moves you one folder up in your current file directory

3.  **Use with caution** delete all the `btec_640` directory you just created:

```
rm -r btec_640/
```

As long as you type the entire dirname you are safe. The flag `-r` is saying remove recursively i.e is going to delete everything inside `btec_640`, always use `rm` with caution.

4. Create again `btec_640` but now without the `-p` flag: `mkdir btec_640`. Repeat step 1 and 2 and see the difference between the files or directories that were created with the `-p` flag and without it.

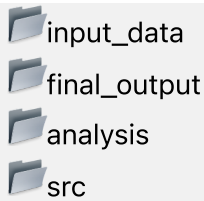
Briefly explain here:

I do not see a difference in using the `-p` flag and not. In `Documents` `mkdir btec_640/class_excercises` and `mkdir -p btec_640/class_excercises` both created a new folder under the `btec_640` folder. mk

5. Go back to `Documents/` and following the **logic** of this command:

```
mkdir -p btec_640/class_excercises: now create two new directories inside
btec_640. One directory for the genomics_project and other for the
```

transcriptomics_project.  Following the structure that we saw in class, **each project directoy** should have inside the following sub-directories:



 copy and paste here the command that you use, **Don't forget to use all the best practices for documentation that we saw in class**

```
bash-3.2$ mkdir -p btec_640/class_exercises/input_data
bash-3.2$ mkdir -p btec_640/class_exercises/final_output
bash-3.2$ mkdir -p btec_640/class_exercises/analysis
bash-3.2$ mkdir -p btec_640/class_exercises/src
bash-3.2$ mkdir -p btec_640/genomics_project/input_data
bash-3.2$ mkdir -p btec_640/genomics_project/final_output
bash-3.2$ mkdir -p btec_640/genomics_project/analysis
bash-3.2$ mkdir -p btec_640/genomics_project/src
bash-3.2$ mkdir -p btec_640/transcriptomics_projects/input_data
bash-3.2$ mkdir -p btec_640/transcriptomics_projects/final_output
bash-3.2$ mkdir -p btec_640/transcriptomics_projects/analysis
bash-3.2$ mkdir -p btec_640/transcriptomics_projects/src
```

From now on we will do our work inside this project. Let's move into it so that our paths stay short and readable.

3. Create your first README file

You will often need to create files from scratch. The simplest way is the command **touch**, which makes an empty file, for example **touch README** will create a file named README.

[!WARNING] Do not reuse filenames in the same directory, otherwise they will be overwritten

Before you create anything, remember: never adding spaces in file or folder names.

To create and edit the README file, go inside the **class_exercises** directory (you should already know how to do it, if you are lost, use **pwd**) and type the following:

```
touch README
```

```
ls #here you should see that the file is there, but it is still empty
```

```
nano README #This will open a "primitive text editor"
```

When nano is open, briefly write a description of this directory, this is where you are going to put all the coding exercises that we do during our classes.

To save and exit type **ctrl + X** and follow the instructions

Congrats! You created your first working directory.

4. Commit and push this file to GitHub

The instructions for this are on canvas